Evaluating the Autonomous Mind: A Multi-Dimensional Framework for Agentic AI Readiness

Fei Wang, Salon Ren, and Eric Wang

***Abstract*—The transition from static large language models to autonomous agentic systems between 2024 and 2026 has exposed a fundamental measurement crisis: evaluation methodologies inherited from traditional natural language processing fail to capture the non-deterministic, multi-step, tool-mediated nature of agentic behavior. Binary success metrics treat agents as black boxes, measuring where they ended up rather than how they got there, a phenomenon we term the *success masking problem*. This paper synthesizes emerging research into a comprehensive evaluation framework built on four pillars: core model, memory, tool use, and environment. It introduces trajectory-level metrics that inspect the complete execution path, addresses the evaluator paradox through agent-as-a-judge paradigms and the Judge Reliability Harness, and proposes the CLEAR framework of cost, latency, efficacy, adherence, and resilience as an enterprise readiness standard. We present Cost-Normalized Accuracy as a reporting requirement and survey the open-source tooling ecosystem that makes these methods practical. The goal is to close the measurement imbalance that currently allows agents to pass benchmarks while failing in production.**

***Index Terms*—Agent-as-a-Judge, agentic AI, Cost-Normalized Accuracy, enterprise readiness, evaluation frameworks, trajectory metrics.**

I. Introduction: The Death of the Black-Box Success Metric

A. The Transition from LLM Chatbots to Agentic Collaborators

Between 2024 and 2026, the enterprise AI landscape underwent a fundamental transformation. The shift from standard large language models to autonomous agentic systems represents not an incremental improvement in capability, but a qualitative change in the computational paradigm itself. Where traditional LLMs operated as sophisticated response engines—accepting a prompt and returning a completion—agentic systems function as persistent, goal-directed entities that perceive, plan, act, and interact with complex environments over extended timeframes.

This transition has been driven by three converging forces. First, the maturation of model capabilities—particularly in reasoning and instruction following—has made it feasible for LLMs to serve as the cognitive core of autonomous systems. Second, the development of standardized tool-use protocols and orchestration frameworks (LangChain, LangGraph, CrewAI, AutoGen) has lowered the engineering barrier to building multi-step agent workflows. Third, enterprise demand for automation that goes beyond content generation—toward actual task execution—has created a commercial imperative for agentic deployment.

The result has been a sharp acceleration in production-grade agentic systems. Chat interfaces, browser-based agents, and enterprise automation platforms have led the surge, with an estimated 87% of Fortune 500 companies piloting or deploying agentic AI systems by Q1 2026. These systems are no longer research prototypes. They are booking flights, executing trades, processing insurance claims, writing and deploying code, and managing customer service interactions—autonomously, and at scale.

B. Defining "Agenticness": Autonomy, Goal-Driven Reasoning, and Scaffolded Intelligence

Not every system that uses an LLM is agentic. The distinction matters because the degree of agency directly determines the complexity of the evaluation challenge. We define agenticness along three axes:

Autonomy The system's capacity to generate tasks, decisions, and strategies without explicit step-by-step human instruction. A conventional AI requires predefined inputs and task boundaries; an agentic system defines its own sub-goals and execution path.

Goal-Driven Reasoning The system's ability to decompose high-level objectives into actionable plans and pursue them across multiple steps, adapting when intermediate results deviate from expectations. This is distinct from single-turn inference, where the model responds to an isolated prompt.

Scaffolded Intelligence The degree to which the system's capability is determined not just by the underlying model, but by the surrounding cognitive architecture: prompt engineering, reflection loops, planning layers, memory management, and tool orchestration. A critical finding from 2025 research is that even when underlying models (like GPT-4 or Claude 3.5) are older or less capable, the agentic scaffolding determines the ultimate capability and productization of the system [1].

These three axes define a spectrum of agenticness. A system that exhibits all three at high levels is fully agentic; one that exhibits some is semi-agentic. The evaluation methodology must scale with the degree of agency. Table I contrasts the two paradigms.

Fei Wang, Salon Ren, and Eric Wang are with Alchedata AI. Corresponding author: Fei Wang.

TABLE I
CONVENTIONAL AI VS. AGENTIC AI ACROSS KEY DIMENSIONS.

Dimension	Conventional AI	Agentic AI
Adaptability	Predefined tasks, static rules	Dynamic, evolving environments
Autonomy	Predefined inputs, boundaries	Self-generated sub-goals
Data Dependency	Fixed training data	Real-time perception & planning
Decision-Making	Deterministic rules	Context-aware, multi-agent
Resource Requirements	Moderate complexity	High orchestration demands

C. The Inadequacy of Current Benchmarks: Non-Determinism and the "Success Masking" Problem

Traditional model evaluation is outcome-based: did the model produce the correct output given this input? For a summarization task or a classification benchmark, this is reasonable. The output is the artifact. There is nothing else to measure.

Agents are different. An agent operates in a continuous decision loop—sensing, planning, acting, and updating state across multiple turns and tool calls. The final output may be correct even when the path to it was broken: an agent can complete a task while violating a safety policy, bypassing an authorization check, or skipping a diagnostic step that would have caught a downstream failure.

Binary success metrics treat the agent as a black box. They measure where it ended up, not how it got there.

This is what researchers now call the **success masking problem**—agents that appear to succeed while simultaneously creating latent risk. It is not a corner case. It is a systematic failure mode of any evaluation methodology that ignores the execution trajectory [2].

Figure 1 illustrates this problem: outcome-only evaluation sees only the final result, while trajectory evaluation reveals the policy violations and skipped steps along the path.

The implications are severe. Organizations are making multi-million dollar infrastructure commitments based on evaluation methodologies that were designed for a fundamentally different paradigm. The result is a growing class of production failures that are entirely invisible to the benchmarks teams rely on.

II. THEORETICAL FOUNDATIONS OF AGENT ASSESSMENT

A. The Four-Pillar Model: Core Model, Memory, Tools, and Environment

Effective agentic evaluation requires decomposing the agent into its functional components and testing each independently. The Agent Assessment Framework (AAF), developed from foundational work at Anthropic and refined in industry deployments, proposes four architectural pillars [1]. These pillars isolate specific components of the agentic scaffold to identify where failures originate—whether in the core model, the memory system, the tool-use logic, or the environmental interaction. Figure 2 provides a visual overview.

1) The Core Model Pillar: Evaluation at the core model level focuses on instruction following and policy alignment. It assesses whether the agent adheres to the intended sequence of objectives and respects established constraints. A critical finding in 2025 research is that even when underlying models are older, the agentic scaffolding—the prompt engineering, reflection loops, and planning layers—determines the ultimate capability and productization of the system [1].

Evaluation at this pillar often involves measuring **calibration error**—the alignment between an agent's confidence and its actual accuracy—which is vital for risk-sensitive workflows. An agent that is consistently overconfident in incorrect outputs is far more dangerous than one that signals uncertainty. Calibration error is measured by comparing the agent's self-assessed confidence scores against its actual success rate across a distribution of tasks.

Key metrics for the core model pillar include:

- **Instruction adherence rate**: The proportion of tasks where the agent follows the specified instruction sequence without deviation.
- **Constraint compliance**: Whether the agent respects boundaries established earlier in a conversation or workflow.
- **Calibration error**: The divergence between confidence estimates and actual accuracy, typically measured using Expected Calibration Error (ECE).
- **Policy alignment**: The degree to which the agent's behavior conforms to stated operational policies.

2) The Memory Pillar: Memory management is evaluated through storage efficiency and retrieval accuracy. This pillar measures the agent's ability to update contextual data without duplication and tracks "update latency"—the time between an environmental change and the agent's incorporation of that change into its working context.

In long-running sessions, agents frequently suffer from **context retention failures**—losing track of constraints established in early turns, a phenomenon sometimes called being "lost in the middle" of the context window. This is not merely an inconvenience; in production workflows, an agent that forgets a constraint established in turn 3 can violate a safety policy in turn 15 without any awareness of the violation.

Metrics such as Precision, Recall, F1-score, and BLEU-1 are employed against gold-standard labels to ensure accurate information extraction from historical context:

- **Retrieval precision**: Of the information the agent retrieved from memory, how much was relevant to the current task?
- **Retrieval recall**: Of the information available in memory that was relevant, how much did the agent actually retrieve?
- **Update accuracy**: When the agent updates its context with new information, how accurately does it represent the change?
- **Duplication rate**: The frequency with which the agent stores redundant or conflicting information in memory.
- **Context retention span**: The number of turns over which the agent maintains accurate awareness of previously established constraints.

3) The Tool Use Pillar: Tool interaction is perhaps the most active area of agentic failure. Evaluation frameworks assess tool selection accuracy, parameter mapping, and tool sequencing. For instance, an agent must not only choose the

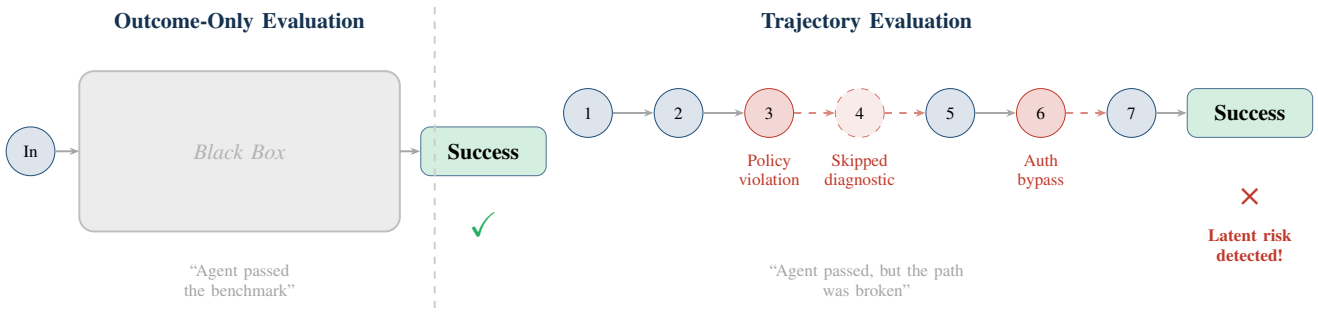

Fig. 1. The success masking problem. Outcome-only evaluation (left) sees only the final result, while trajectory evaluation (right) reveals policy violations and skipped steps along the execution path.

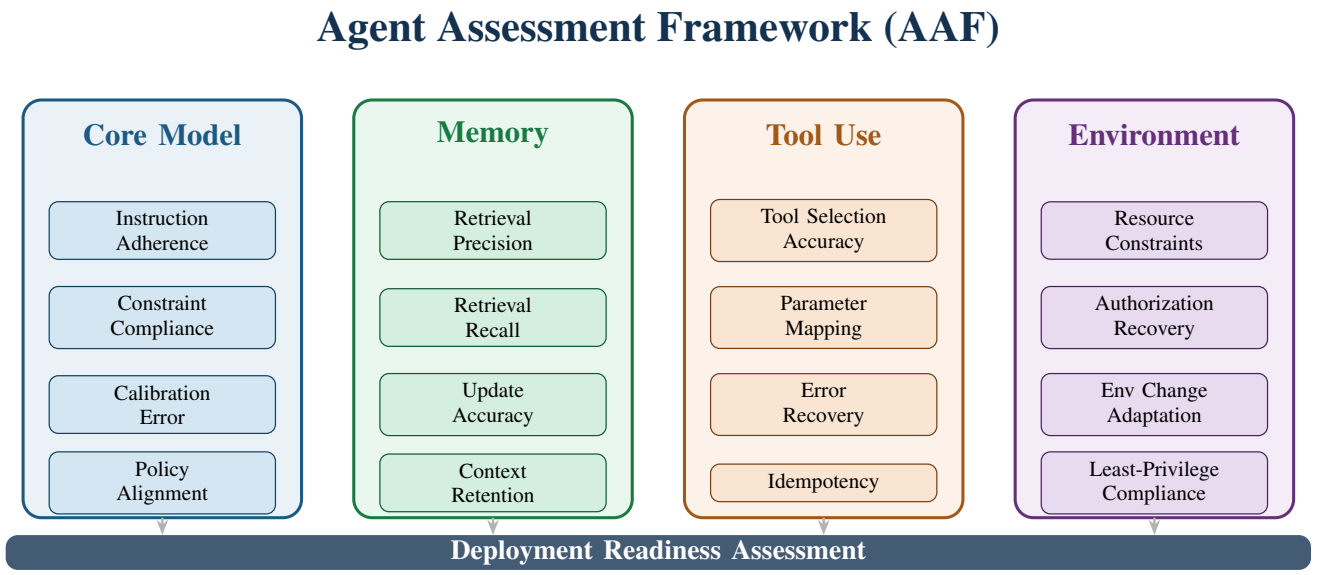

Fig. 2. The Four-Pillar Model for Agent Assessment. Each pillar isolates a functional component of the agentic scaffold, enabling targeted evaluation and diagnosis of failure origins.

correct tool (e.g., a monitoring tool vs. an audit tool) but must also pass semantically accurate parameters, such as using an Instance ID rather than a Region Name.

Production-grade agents must demonstrate resilience by recognizing and recovering from invalid tool invocations, malformed parameters, and unexpected response formats. The tool use pillar evaluates:

- **Tool selection accuracy**: Whether the agent chooses the appropriate tool for each sub-task.
- **Parameter mapping fidelity**: Whether the agent passes correct, semantically appropriate parameters to each tool.
- **Tool sequencing**: Whether the agent calls tools in the correct order when dependencies exist.
- **Error recovery**: The agent's ability to recognize and handle invalid invocations and malformed responses without cascading failure.
- **Idempotency**: Whether the agent can handle retries safely without causing unintended side effects, such as duplicating a purchase or a data entry.

4) The Environment Pillar: Environmental evaluation examines how the agent responds to resource limitations, authorization failures, and changes in environmental constraints. It assesses the agent's ability to preserve intended workflows within real or simulated operational contexts. Secure operation in these environments requires validating authentication, enforcing role-based access controls, and limiting agent access based on least-privilege principles.

The environment pillar is particularly important because it tests the agent's behavior under conditions that are difficult to replicate in static test sets:

- **Resource constraint handling**: How the agent behaves when API rate limits are hit, memory is constrained, or compute resources are limited.
- **Authorization failure recovery**: Whether the agent gracefully handles permission errors and seeks alternative approaches rather than retrying indefinitely.
- **Environmental change adaptation**: Whether the agent detects and adapts when the environment changes.
- **Least-privilege compliance**: Whether the agent restricts its own actions to the minimum necessary permissions.

Testing across all four pillars is what separates a benchmark score from a deployment readiness assessment.

B. Static Verification vs. Dynamic Execution Monitoring

The assessment of agentic AI has moved beyond static test sets toward dynamic, interactive, and judge-based methodologies that capture the fluid nature of agent-environment interactions.

Static analysis validates agent behavior against predefined ground-truth specifications and "golden labels." This approach is necessary but insufficient. It confirms that the agent produces correct outputs for known inputs, but it cannot capture the full range of behaviors that emerge during actual execution.

However, many failures in multi-agent systems only surface during dynamic execution. Dynamic analysis involves monitoring runtime behaviors to detect deviations, policy violations, and guardrail breaches during actual interaction with tools and environments. This approach allows for the measurement of **trajectory metrics**, which evaluate the complete execution path—every reasoning step and tool call—rather than just the final output.

The distinction is fundamental, as summarized in Table II.

TABLE II
STATIC VERIFICATION VS. DYNAMIC EXECUTION MONITORING.

Aspect	Static Verification	Dynamic Monitoring
What is measured	Output vs. golden labels	Complete execution trajectory
When measured	Pre-deployment, curated test sets	During execution, real/simulated env
Failure modes	Incorrect outputs	Process failures, policy violations
Adaptability	Fixed test set	Responsive to env changes
Scalability	High (automated)	Moderate (requires instrumentation)
Coverage	Known scenarios	Emergent behaviors

Effective evaluation requires both. Static verification provides a baseline; dynamic monitoring reveals the behaviors that matter in production.

C. The Socio-Technical Scenario Manifold: Beyond Benchmark Islands

A fundamental limitation of current evaluation practice is its reliance on what researchers call "benchmark islands"—disconnected instances of task completion that fail to represent the full distribution of conditions an agent will encounter in deployment.

The Holographic Agent Assessment Framework (HAAF) proposes a move from benchmark islands to a **scenario manifold** that characterizes agent trustworthiness over a representative socio-technical distribution [2]. The framework models each scenario $s \in \mathcal{S}$ as a structured configuration varying along axes such as task objective, tool interface, and social context. The framework produces a vector of trustworthiness measurements $\mathbf{m}(a, s)$, which are then aggregated over a weighted test set $Q \subset \mathcal{S}$ to yield an estimated trustworthiness profile:

$$\hat{\mathbf{T}}_Q(a) = \frac{1}{Z} \sum_{s \in Q} w(s)\, \mathbf{m}(a, s) \tag{1}$$

where $w(s)$ encodes deployment relevance and risk sensitivity, and Z is a normalization factor. This allows for a risk-aware assessment where high-consequence scenarios are upweighted.

The key insight is that not all scenarios are equally important. An agent that performs well on low-stakes tasks but fails on high-stakes ones is not trustworthy, even if its average performance across all scenarios appears acceptable. The scenario manifold approach enables organizations to weight evaluation by the consequences of failure, producing an assessment that reflects real-world risk.

This represents a paradigm shift from "how often does the agent succeed?" to "how trustworthy is the agent in the scenarios that matter most?"

III. Cognitive Diagnostics: Probing the Reasoning Trace

A. Repository-Level Reasoning and the Aggregation Deficit

As agentic systems are deployed on increasingly complex tasks—particularly in software engineering—evaluation must probe not just whether the agent produces correct output, but whether it can reason effectively about large, interdependent code systems.

RepoReason is a diagnostic benchmark designed to evaluate repository-level reasoning in LLM agents [3]. It moves away from code generation to a **verification-centric** approach called Abductive Assertion Verification. By masking unit test assertions and requiring the model to derive values that satisfy them, RepoReason tests the agent's ability to mentally reconstruct execution history across massive, interdependent file systems.

This approach is significant because it directly tests the kind of reasoning that production agents must perform: understanding how changes in one file propagate through dependencies, predicting the behavior of code without executing it, and synthesizing information across multiple modules.

Findings from RepoReason have identified critical performance ceilings for current frontier models that have direct implications for deployment:

The Cliff Effect Reading comprehension accuracy drops sharply when the volume of code exceeds approximately 600 lines. This is not a gradual decline but a precipitous drop—agents that perform well on 500-line codebases may fail catastrophically on 700-line ones.

The Aggregation Deficit Accuracy declines as "integration width" increases—meaning the number of cross-file dependencies the agent must synthesize. This is precisely the condition that characterizes real production codebases, where changes in one module can affect behavior in seemingly unrelated components.

Consistency Decay Models show a significant loss of consistency beyond 100 execution steps in a single trajectory. An agent that reasons correctly in steps 1–80 may begin making errors in steps 80–120, not because the later steps are harder, but because the accumulated context degrades the agent's reasoning coherence.

These are not model-specific quirks. They reflect fundamental constraints on how current architectures handle deep, multi-file reasoning at scale. Any evaluation methodology that does not test for these failure modes will produce overconfident deployment assessments.

B. The "Cliff Effect" and Simulation Depth Limits

The Cliff Effect deserves particular attention because it represents a qualitative rather than quantitative failure mode. Unlike gradual performance degradation, the Cliff Effect means that an agent can appear fully competent right up to the point where it catastrophically fails.

This has profound implications for evaluation:

1) **Interpolation is dangerous.** If an agent performs well on 400-line codebases and poorly on 800-line ones, it is not safe to assume it performs acceptably on 600-line codebases. The cliff may occur at any point.
2) **Benchmark results can be misleading.** A benchmark that uses 500-line codebases will produce optimistic results that do not generalize to the 700+ line codebases common in production.
3) **Safety margins must be explicit.** Organizations must define maximum complexity thresholds for their agents and enforce them in production, rather than discovering the cliff through failures.

The simulation depth limit—the point at which an agent's reasoning coherence degrades beyond recovery—is similarly consequential. For agents operating in multi-turn workflows, the 100-step consistency decay means that long-running tasks are inherently less reliable than short ones. This must be accounted for in both evaluation and deployment design:

- Break long workflows into shorter segments with verification checkpoints.
- Implement intermediate validation steps that can catch reasoning drift before it compounds.
- Monitor trajectory coherence in real-time and escalate to human oversight when degradation is detected.

C. Agent GPA: Decomposing the Goal-Plan-Action Cycle

Traditional evaluation treats task completion as a monolithic outcome. But agentic behavior can be decomposed into three distinct cognitive phases, each of which can fail independently [3]:

1) **Goal comprehension**: Does the agent correctly understand what it is being asked to do?
2) **Plan formulation**: Does the agent devise a sensible strategy for achieving the goal?
3) **Action execution**: Does the agent correctly implement the plan through tool calls and environmental interactions?

The **Agent GPA** (Goal-Plan-Action) framework evaluates each phase independently, producing a profile rather than a single score. Figure 3 illustrates this decomposition.

This decomposition is critical because the remediation for each type of failure is fundamentally different, as shown in Table III.

An agent with a high Goal score but low Action score understands what to do but cannot execute it. An agent with a high Action score but low Goal score can execute flawlessly—but on the wrong task. These are fundamentally different problems requiring fundamentally different solutions.

TABLE III
GPA FAILURE PHASES, SYMPTOMS, ROOT CAUSES, AND REMEDIATIONS.

Phase	Symptom	Root Cause	Remediation
Goal	Solves wrong problem	Misunderstood intent	Prompt eng., clarification
Plan	Suboptimal path	Insufficient reasoning	Reflection loops, verification
Action	Execution errors	Tool use failures	Better descriptions, validation

IV. The Evaluator Paradox: Reliability in Automated Judgment

A. LLM-as-a-Judge vs. Agent-as-a-Judge: Hierarchical vs. Monolithic Assessment

As tasks become more open-ended and nuanced, Large Language Models are increasingly used as evaluators to assess qualities like helpfulness, coherence, and faithfulness that resist mechanical checking. This is a practical necessity—human labeling at evaluation scale is not viable for most organizations.

This is typically implemented in two paradigms:

LLM-as-a-Judge employs a monolithic model to perform a single-pass qualitative assessment of execution logs and outputs using structured prompts and rubrics [4]. It is fast, scalable, and relatively inexpensive. However, it is limited by the judge's ability to process complex, multi-step execution traces in a single inference pass, and it is susceptible to systematic biases.

Agent-as-a-Judge employs a specialized "auditor agent" or decentralized team of agents that evaluates a subject agent using tool calls, code execution, and environment interaction rather than linguistic plausibility alone [5]. The auditor agent can decompose complex evaluation goals into sub-tasks, verify claims through actual execution, and provide fine-grained feedback on specific steps in the execution trace.

Research published in 2026 demonstrates that Agent-as-a-Judge aligns with human experts approximately **90% of the time**, significantly outperforming the **70% alignment rate** of traditional LLM-as-a-Judge methods. The gap is explained by the auditor agent's ability to decompose complex evaluation goals into sub-tasks and verify claims through actual execution. Figure 4 contrasts the two paradigms.

The key differences are summarized in Table IV.

TABLE IV
LLM-AS-A-JUDGE VS. AGENT-AS-A-JUDGE ACROSS KEY DIMENSIONS.

Feature	LLM-as-a-Judge	Agent-as-a-Judge
Evaluation Mode	Single-pass inference	Autonomous, hierarchical
Verification Basis	Linguistic plausibility	Execution & interaction
Robustness	Prone to parametric biases	Decentralized deliberation
Intermediate Feedback	Limited or absent	Rich step-level feedback
Human Alignment	$\sim$70%	$\sim$90%
Cost	Lower	Higher
Latency	Faster	Slower

The Agent-as-a-Judge paradigm addresses the cognitive overload faced by monolithic judges when assessing multi-step, complex tasks. By decomposing evaluation goals into sub-tasks and utilizing tools like code interpreters to verify the agent's actions, agentic judges can provide much finer-grained feedback and pinpoint specific flaws that might be obscured in a global score.

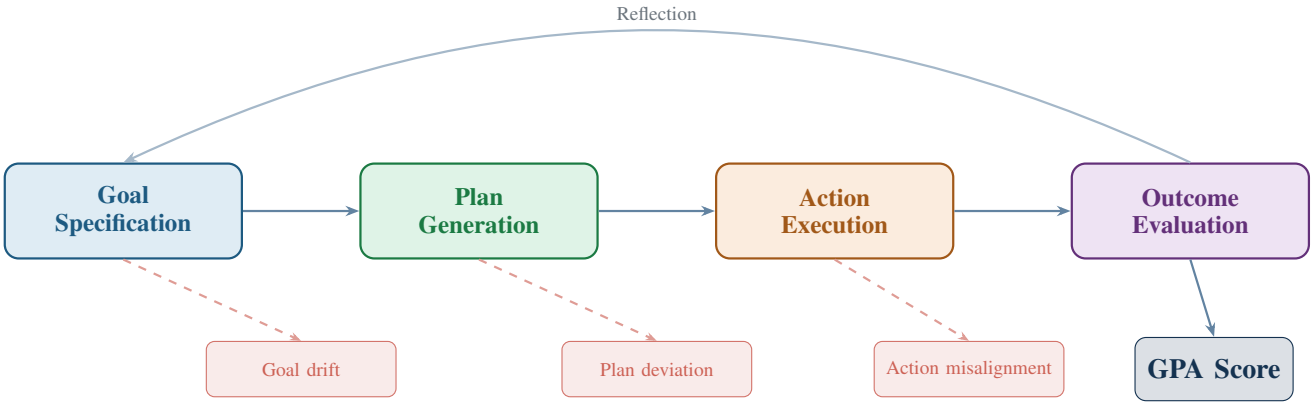

Fig. 3. The Agent GPA framework decomposes agentic behavior into Goal comprehension, Plan formulation, and Action execution, with failure modes identified at each phase and a composite GPA score reflecting overall alignment.

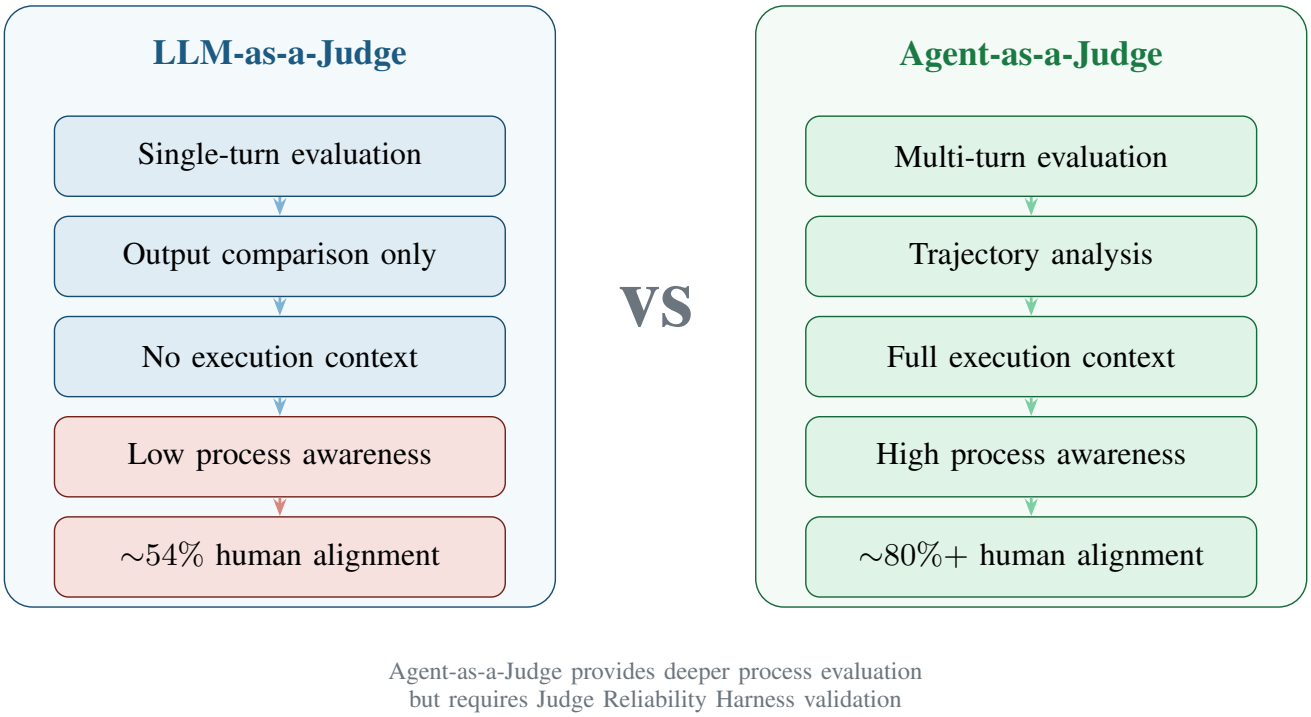

Fig. 4. LLM-as-a-Judge vs. Agent-as-a-Judge comparison. Agent-as-a-Judge provides deeper process evaluation through multi-turn analysis and trajectory inspection, achieving significantly higher human alignment.

For production-critical workflows, the question is not just "what did the agent do?" but "can we trust the system that's checking what the agent did?"

B. Taxonomy of Judge Biases: Verbosity, Family, and Position Biases

LLM judges introduce their own failure modes, and these must be understood and mitigated for evaluation results to be trustworthy. Research from the Judge Reliability Harness (JRH) identifies several systematic biases [6]:

Verbosity Bias — Judges over-reward longer responses independent of quality. An agent that produces a verbose, meandering response may receive a higher score than one that produces a concise, accurate answer. This bias is particularly dangerous because it creates a perverse incentive: agents optimized against verbose judges will learn to pad their outputs, increasing token costs without improving quality.

Position Bias — Judges favor responses appearing earlier in a comparison. When evaluating multiple agent outputs side by side, the judge systematically prefers the first option presented, regardless of actual quality. This bias undermines the validity of comparative evaluations and leaderboard-style rankings.

Family Bias — Judges favor outputs from the same model provider. A GPT-based judge may systematically prefer GPT-generated outputs, and a Claude-based judge may prefer Claude-generated outputs. This creates a conflict of interest when the judge and the agent share the same model family.

Format Sensitivity — Formatting changes produce larger

reliability drops than semantic ones. A judge may assign different scores to substantively identical outputs that differ only in formatting (bullet points vs. paragraphs, markdown vs. plain text). This means that evaluation results can be manipulated by formatting choices rather than quality improvements.

These biases are not theoretical. They have been empirically demonstrated across multiple judge models and evaluation contexts. Any organization using LLM-based evaluation must account for them.

C. The Judge Reliability Harness: Perturbation-Based Stress Testing

The Judge Reliability Harness (JRH) is an open-source library developed to stress-test LLM evaluators [6]. It generates reliability tests using various perturbations to quantify how much a judge's scores should actually be trusted.

JRH employs four perturbation strategies:

Label Flip — Rewriting responses to clearly violate rubrics to test discriminative accuracy. If a judge cannot reliably identify a response that has been deliberately modified to violate the evaluation criteria, the judge's discriminative ability is questionable.

Format Invariance — Altering visual layouts and formatting to ensure the judge is not distracted by non-semantic changes. A reliable judge should assign the same score to substantively identical content regardless of formatting.

Semantic Paraphrase — Changing wording while maintaining meaning to test scoring stability. If a judge assigns significantly different scores to paraphrased versions of the same content, the judge is evaluating surface form rather than substance.

Verbosity Bias Tests — Testing whether judges over-reward longer answers when quality is held constant. By presenting the same content at different levels of verbosity, JRH can quantify the magnitude of verbosity bias.

Experiments with JRH indicate that formatting perturbations often produce larger reliability drops than semantic ones, and that judge performance in free-response tasks does not necessarily generalize to agentic settings. This is a critical finding: a judge that performs reliably on traditional NLP benchmarks may be unreliable when evaluating agentic behavior.

Recommendations for practitioners:

1) **Stress-test your judges before trusting them.** Run JRH-style perturbations against any LLM judge before relying on its outputs for production decisions.
2) **Use multiple judges for high-stakes evaluations.** Employ 2–3 different judge models and require consensus or majority agreement.
3) **Calibrate against human judgments.** Before deploying LLM-as-judge at scale, calibrate it against human evaluations on at least 100 examples.
4) **Separate evaluation from generation.** Never use the same model to both generate agent behavior and evaluate it.
5) **Implement judge auditing.** Regularly sample and review judge evaluations to detect drift, bias, or degradation.

V. Operationalizing Agency: Economics, Safety, and Latency

A. The Unreliability Tax and the Cost of Autonomy

Agentic AI introduces a fundamentally different cost model from traditional software infrastructure. Infrastructure costs are relatively fixed; intelligence costs are variable and correlated with agent complexity. When agents fail and retry, costs compound quickly.

Research into the **Unreliability Tax** identifies the additional compute, latency, and engineering required to compensate for agent failures [7]. For instance, a "Reflexion" loop that runs for 10 cycles can consume **50 times the tokens** of a single linear pass. This is not a marginal increase—it represents a qualitative change in the economics of AI deployment.

The Unreliability Tax manifests in several ways:

- **Retry costs**: When agents fail, they must retry, consuming additional tokens and API calls. In complex workflows, a single failure can trigger a cascade of retries across multiple steps.
- **Guardrail costs**: Implementing safety checks, validation steps, and human-in-the-loop checkpoints adds latency and token consumption to every workflow.
- **Observability costs**: Capturing and storing execution traces for evaluation and debugging requires infrastructure and storage.
- **Engineering costs**: Building and maintaining the evaluation infrastructure, red teaming pipelines, and monitoring systems represents a significant ongoing investment.

These costs are often invisible in benchmark evaluations, which typically measure accuracy in isolation. In production, they can be the difference between a viable product and an economic failure.

B. CNA (Cost-Normalized Accuracy) as a New Industry Standard

To evaluate economic efficiency, the metric of **Cost-Normalized Accuracy (CNA)** is used [8]:

$$\text{CNA} = \frac{\text{Accuracy}}{\text{Cost}} \times 100 \tag{2}$$

where Cost is measured in USD per task. CNA makes the accuracy-cost tradeoff explicit, enabling organizations to compare agents not just on how well they perform, but on how efficiently they achieve that performance.

Evaluation of leading agent architectures on enterprise tasks demonstrates a striking result, shown in Table V and visualized in Figure 5.

TABLE V
Architecture comparison: efficacy, cost, and CNA.

Architecture	Efficacy (%)	Cost (USD/Task)	CNA
ReAct-GPT-o3	68.7	0.85	80.8
Reflexion	74.1	4.35	17.0
Domain-Tuned	81.5	0.31	260.4
Plan-Execute	71.9	1.05	68.5

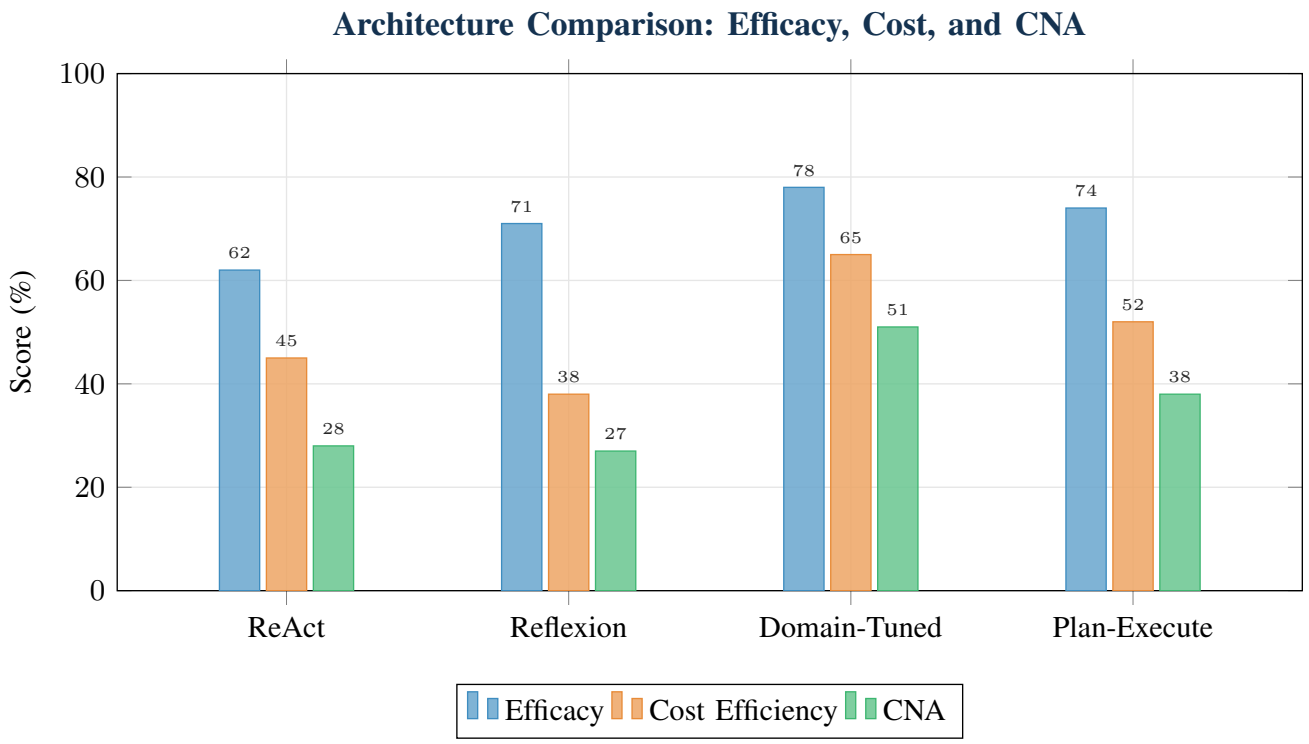

Fig. 5. Architecture comparison across efficacy, cost efficiency, and Cost-Normalized Accuracy (CNA). The Domain-Tuned architecture achieves the highest CNA by combining strong efficacy with low cost, while Reflexion's high efficacy is undermined by excessive cost.

The Reflexion architecture—often cited for its high accuracy—is **4.7× more expensive** than the domain-tuned alternative while delivering lower accuracy. The Domain-Tuned agent achieves the highest accuracy at the lowest cost, producing a CNA more than 15× higher than Reflexion.

This finding highlights what researchers call a **distorted research landscape** where expensive and fragile solutions appear superior in standard accuracy-only benchmarks [7]. Simple baseline strategies can sometimes outperform complex agents at 50× lower cost.

CNA should be a standard reporting requirement alongside any accuracy metric. Any benchmark result reported without cost normalization is incomplete. The most accurate agent in an evaluation suite may be economically inviable at production scale.

C. Security Challenges: Offensive Agentic Risk and API/MCP Gateways

As agentic systems gain the ability to take autonomous actions, the security implications become critical [9]. The attack surface of agentic environments maps across four layers, as illustrated in Figure 6.

1) **The Endpoint Layer** (e.g., coding agents): Compromise of the agent's execution environment, including local file systems, development environments, and developer machines.
2) **The API/MCP Gateway Layer** (tool exchanges): Manipulation of the tool interfaces that agents use to interact with external systems. A compromised API can feed the agent malicious data, causing it to take harmful actions.
3) **The SaaS Platform Layer**: Exploitation of the SaaS applications that agents interact with—CRM systems, communication platforms, cloud services—through the agent's authorized access.
4) **The Runtime Layer**: Attacks on the agent's execution runtime, including prompt injection, context manipulation, and goal hijacking.

CISOs are encouraged to implement a three-stage security framework:

- **Visibility**: Knowing what agents you have, what tools they can access, and what actions they are taking. You cannot secure what you cannot see.
- **Configuration**: Reducing the blast radius by implementing least-privilege access, tool restrictions, and action boundaries. Agents should have the minimum permissions necessary for their tasks.
- **Runtime Protection**: Monitoring agent behavior in real-time for anomalous actions, policy violations, and indicators of compromise.

The security challenge is compounded by the fact that agentic systems can be manipulated through their inputs in ways that traditional software cannot. Prompt injection—embedding malicious instructions in data that the agent processes—can cause agents to take actions that violate their intended behavior. Tool poisoning—manipulating the outputs of tools that the agent relies on—can cause agents to make incorrect decisions based on falsified information.

These security considerations must be integrated into evaluation from the beginning, not bolted on after deployment. An

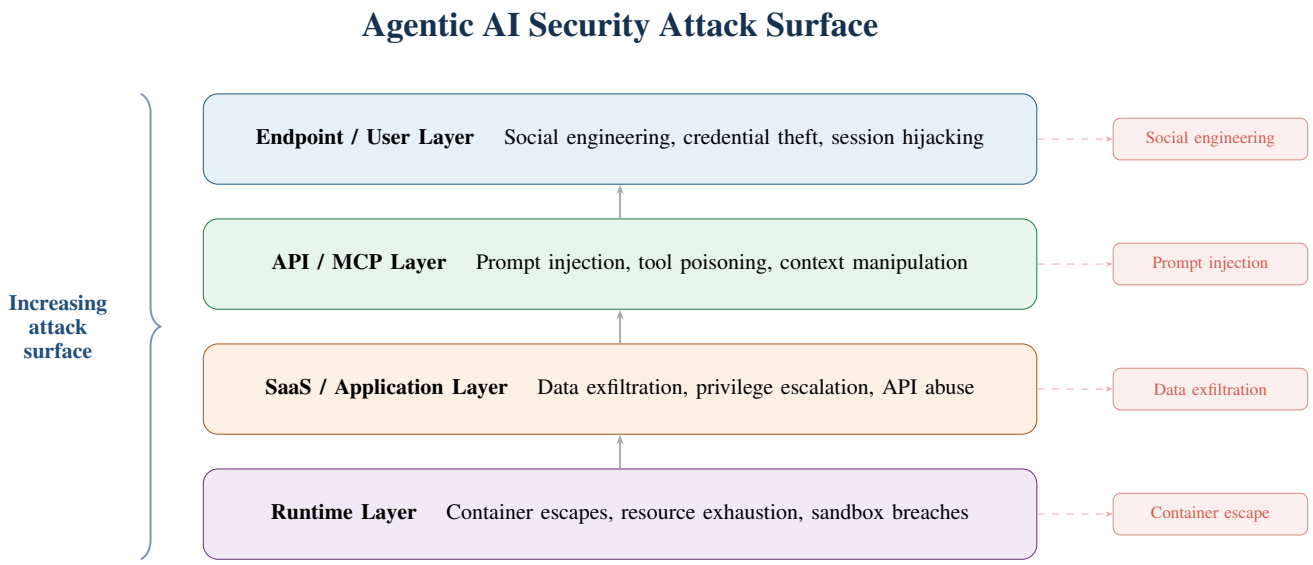

Fig. 6. The four-layer security attack surface for agentic AI systems, from runtime infrastructure to endpoint users, with representative threat vectors at each layer.

evaluation framework that does not test for security vulnerabilities is providing a false sense of security.

VI. From Lab to Production: The Enterprise Readiness Standard

A. The CLEAR Framework for Deployment Governance

For organizations to move beyond pilots, they require a holistic evaluation across what is known as the **CLEAR dimensions** [8]:

C — Cost per task (absolute and normalized): Not just the raw cost of API calls and compute, but the cost normalized by task complexity and success rate. CNA provides the standard metric.

L — Latency (time to first token, end-to-end response): Not just average latency, but the full distribution—including P95 and P99 latency, which determine whether the agent meets SLA requirements. Time-to-first-token (TTFT) and total response time both impact real-time usability.

E — Efficacy and accuracy (trajectory-level, not outcome-only): Moving beyond binary success metrics to evaluate the complete execution path. This includes the Agent GPA decomposition (Goal-Plan-Action) and trajectory coherence metrics.

A — Adherence and reliability (policy compliance, idempotency): Whether the agent respects established constraints, follows operational policies, and handles retries safely without causing unintended side effects. Constraint violation rates and idempotency testing are essential.

R — Resilience and stability (failure recovery, long-session consistency): The agent's ability to recover from errors, maintain performance over long sessions, and degrade gracefully under adverse conditions rather than failing catastrophically.

Figure 7 visualizes the CLEAR framework as a radar chart, comparing an ideal agent profile against a typical production agent.

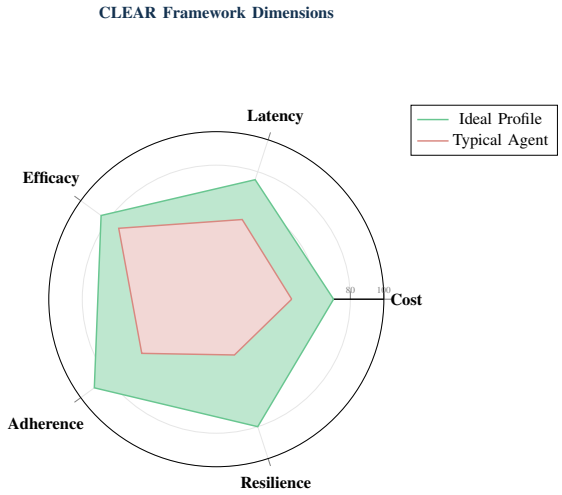

Fig. 7. The CLEAR framework radar chart, comparing an ideal agent profile (high scores across all dimensions) against a typical production agent (with notable gaps in Adherence and Resilience).

No single dimension is sufficient. An agent that scores well on accuracy but fails on adherence is not enterprise-ready. An agent that is fast but unreliable is not deployable. The CLEAR framework requires assessment across all five dimensions before deployment.

B. Simulation-Based Testing: Synthetic Users and Multi-Turn Stability

A significant challenge in agent development is proving readiness for production before the agent encounters real users. Tools like **ArkSim** have emerged to facilitate multi-turn conversation simulation between agents and synthetic users [10].

ArkSim, an open-source framework designed for LangChain and LangGraph agents, integrates evaluations into the de-

veloper workflow via CI/CD platforms. This allows for the detection of regressions and failures early in the lifecycle. Multi-turn simulation is particularly effective for testing:

- **Context loss during long interactions**: Whether the agent maintains awareness of constraints and information established in early turns.
- **Unexpected conversation paths**: Scenarios that emerge only after several turns of interaction and would not be captured by single-turn evaluation.
- **Idempotency**: The ability of an agent to handle retries safely without causing unintended side effects, such as duplicating a purchase or a data entry.
- **Edge cases in multi-turn reasoning**: Reasoning failures that only emerge when the agent must synthesize information across multiple turns.

Simulation-based testing bridges the gap between static benchmarks and production reality. By generating synthetic user interactions that approximate the diversity and unpredictability of real users, simulation enables teams to find issues before users do.

C. Human-in-the-Loop: Calibrating Automated Evaluators for High-Stakes Domains

Human-in-the-loop (HITL) processes remain essential to audit evaluation results and ensure reliability [11]. Humans provide the ground truth labels for "golden testing datasets" used to verify agent-generated intents. However, human evaluation at scale is not economically viable for most organizations.

The practical approach is a **layered evaluation strategy**:

1) **Automated evaluation** for structured, verifiable outputs: Format compliance, factual accuracy against known sources, constraint violation detection.
2) **LLM-as-a-Judge** for scalable quality assessment: Reasoning coherence, output quality, decision appropriateness—calibrated against human judgments.
3) **Agent-as-a-Judge** for complex, multi-step evaluations: Where fine-grained feedback on specific reasoning steps and tool interactions is required.
4) **Human evaluation** for calibration and high-stakes decisions: Periodic review of automated evaluation results, assessment of ambiguous cases, and final approval for high-stakes deployments.

Effective tooling must prioritize the **"right" traces for human review**—such as anomaly signals or low-confidence scores—rather than random sampling. Not all traces are equally informative; human attention should be directed toward the cases where automated evaluation is least confident or most surprising.

This layered approach ensures that human expertise is deployed where it adds the most value, while automated methods handle the volume of evaluation that humans cannot.

VII. The Evaluation Tooling Landscape: Open Source Packages and Industry Platforms

The methodological frameworks described in the preceding sections require practical tooling to be adopted at scale. Between 2024 and 2026, a rapidly maturing ecosystem of open source packages and industry platforms has emerged to address the evaluation gap. This section surveys the current landscape, organized by function, and identifies the strengths and limitations of each category.

A. Open Source Evaluation and Simulation Packages

The open source community has been instrumental in advancing agentic evaluation, particularly in areas where proprietary tools lag behind—namely, multi-turn simulation, multi-agent orchestration, and benchmark automation.

1) ArkSim (arklexai/arksim): ArkSim is an open-source framework designed specifically for multi-turn conversation simulation between agents and synthetic users [10]. Built for LangChain and LangGraph agents, it integrates evaluations into the developer workflow via CI/CD pipelines, enabling the detection of regressions and failures early in the development lifecycle.

ArkSim addresses a critical gap in the evaluation landscape: the inability to test agents under realistic multi-turn conditions before deployment. It generates synthetic user personas that interact with the agent across extended conversations, surfacing context loss, idempotency failures, and unexpected conversational paths that only appear after several turns. Key capabilities include:

- **Synthetic user generation**: Configurable personas that simulate diverse user behaviors and edge cases.
- **Multi-turn conversation simulation**: Extended interaction sequences that test context retention and reasoning coherence over time.
- **CI/CD integration**: Automated regression testing that catches failures before they reach production.
- **Idempotency testing**: Verification that agent retries do not cause unintended side effects such as duplicate transactions or data entries.

2) AgentScope (agentscope-ai/agentscope): AgentScope is a production-ready framework with built-in support for multi-agent orchestration and observability [12]. Unlike single-agent evaluation tools, AgentScope is designed to evaluate the behavior of systems where multiple agents interact—surfacing emergent behaviors, coordination failures, and communication inefficiencies that are invisible when each agent is tested in isolation.

The framework provides multi-agent orchestration tools, built-in observability with comprehensive tracing, distributed execution support, and flexible configuration for diverse agent architectures. AgentScope is particularly relevant for organizations deploying multi-agent systems, where the evaluation challenge is qualitatively different from single-agent assessment.

3) Evaluation-Agent (Vchitect/Evaluation-Agent): The Evaluation-Agent framework, developed by the Vchitect team and recognized with an ACL 2025 Oral and Award, is tailored for evaluating visual generative models using human-like, multi-round strategies [4]. It represents a domain-specific application of the Agent-as-a-Judge paradigm described in Section IV.

Rather than relying on single-pass scoring, the EvaluationAgent engages in multi-round evaluation—asking follow-up questions, requesting clarifications, and progressively refining its assessment. This mirrors the way human evaluators approach complex judgments, and has been shown to produce evaluations that align more closely with human expert assessments.

4) BenchAgents: BenchAgents automates the creation of evaluation benchmarks using LLM-agent orchestration [13]. Rather than requiring human experts to manually curate test sets, BenchAgents uses a team of LLM agents to generate, validate, and refine evaluation benchmarks automatically. The framework addresses a fundamental bottleneck in agentic evaluation: the scarcity of high-quality, domain-specific test sets.

B. Industry Platforms and Frameworks

Alongside open source tools, several industry platforms have emerged that provide managed evaluation infrastructure for enterprise deployments.

1) AWS Strands Evals: AWS Strands Evals provides a practical guide for judgmental evaluation using natural language rubrics and experiments [14]. It enables teams to define evaluation criteria in plain English, run structured experiments, and compare agent performance across configurations.

2) Amazon Bedrock AgentCore Evaluations: Amazon Bedrock AgentCore Evaluations provides automated assessment tools for maintaining consistency across contexts [15]. It is designed for teams deploying agents on the AWS Bedrock platform, and integrates evaluation into the agent development and deployment workflow.

3) LangSmith (LangChain): LangSmith provides tracing and evaluation templates for LangChain-native workflows [10]. As the most widely adopted agent orchestration framework, LangChain's evaluation tooling is particularly important: it provides the infrastructure for capturing execution traces, defining evaluation criteria, and running structured evaluations. LangSmith's tracing capabilities are a practical implementation of the trajectory observability that this paper argues is a prerequisite for meaningful evaluation.

C. Selecting the Right Tooling: A Decision Framework

The choice of evaluation tooling should be driven by the specific evaluation needs of the organization, not by tool availability. Table VI maps evaluation requirements to tool categories.

TABLE VI
EVALUATION NEEDS MAPPED TO RECOMMENDED TOOL CATEGORIES.

Evaluation Need	Tool Category	Example Tools
Multi-turn testing	Simulation frameworks	ArkSim
Multi-agent evaluation	Orchestration + observability	AgentScope
Visual/multimodal eval	Domain-specific judges	Evaluation-Agent
Benchmark creation	Automated generators	BenchAgents
Enterprise readiness	Managed platforms	Strands Evals
Trajectory observability	Tracing frameworks	LangSmith
Judge reliability	Perturbation testing	JRH

No single tool addresses all evaluation needs. Effective evaluation infrastructure typically combines multiple tools—using simulation for pre-deployment testing, tracing for production observability, and judge reliability testing for evaluation quality assurance. The key principle is that evaluation tooling should be selected to match the evaluation methodology, not the other way around.

VIII. Future Directions: Closing the Measurement Imbalance

A. Integrating Human-Centered and Economic Metrics

A systematic review of papers from 2023 to 2025 reveals a **measurement imbalance**: 83% of evaluations focus on technical performance, while only 30% consider human-centered factors and 30% consider economic impacts [7]. This gap creates a fundamental disconnect between benchmark success and deployment value, often leading to rollout reversals and failed investments. Figure 8 visualizes this imbalance.

The measurement imbalance is not merely an academic concern. It has direct business consequences:

- Organizations invest in agents that perform well on technical benchmarks but fail to deliver value to users.
- Procurement decisions are made based on accuracy metrics that do not account for cost, latency, or reliability.
- Agents are deployed that are technically capable but operationally fragile, leading to production incidents and erosion of trust.

Closing this imbalance requires three shifts:

1) **Mandatory economic reporting**: Every benchmark result should include CNA or an equivalent cost-normalized metric. Accuracy without cost context is misleading.
2) **Human-centered evaluation integration**: User satisfaction, task completion from the user's perspective, and the quality of the human-agent interaction should be standard evaluation dimensions, not afterthoughts.
3) **Multi-dimensional deployment criteria**: The CLEAR framework (or equivalent) should be adopted as a standard for deployment readiness, requiring assessment across all five dimensions.

B. The Role of Standardized Web Conduct and Multi-Agent Interaction Protocols

As agentic AI matures, the industry is moving toward a more nuanced understanding of "agenticness" as a spectrum rather than a binary property [16]. Systems are increasingly being designed with a **flexible thinking budget**—skipping complex orchestrators for reactive tasks and allocating more tokens and time for deep reasoning tasks like supply chain planning.

This evolution creates new evaluation challenges:

- **Multi-agent evaluation**: When multiple agents interact, emergent behaviors can arise that are not predictable from evaluating each agent in isolation. Metrics for coordination, communication efficiency, and collective decision-making are needed.

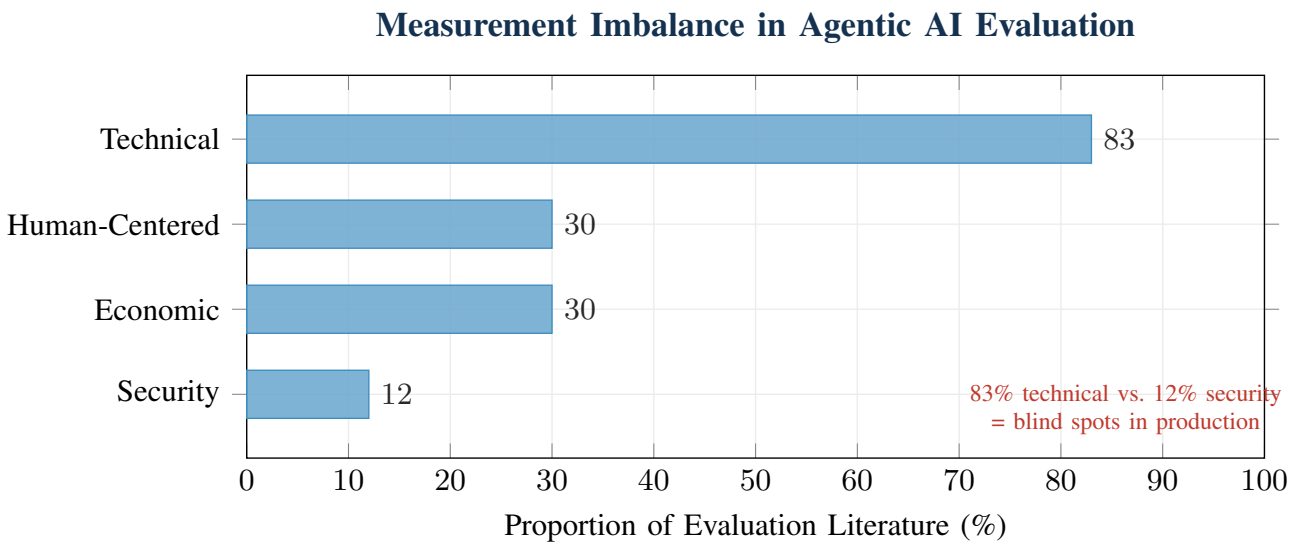

Fig. 8. The measurement imbalance in agentic AI evaluation. Technical metrics dominate the literature, while security and human-centered considerations remain critically underrepresented.

- **Standardized interaction protocols**: As agents interact with web services, APIs, and each other, standardized protocols for agent conduct are needed—analogous to robots.txt for web crawlers, but for autonomous AI agents.
- **Cross-platform benchmarking**: Current benchmarks are fragmented and non-comparable. Industry-standard evaluation protocols are needed to enable meaningful comparison across agents and platforms.

The development of these standards will require collaboration across industry, academia, and regulatory bodies. Organizations that contribute to standardization efforts will have a voice in shaping the evaluation landscape.

C. Toward Verifiable and Trustworthy Autonomous Agents

The 2026 target for enterprise-grade agentic systems is **95%+ task accuracy** in multi-turn reasoning workflows while maintaining economic sustainability [17]. Reaching that target requires closing the measurement imbalance—integrating technical performance, human-centered assessment, and economic efficiency into a single evaluation standard.

The methodological building blocks now exist:

- **Trajectory metrics** that evaluate the complete execution path, not just the final output.
- **Agent-as-a-Judge** paradigms that provide scalable, human-aligned evaluation.
- **CNA** and the CLEAR framework that integrate economic and operational dimensions.
- **Simulation-based pre-deployment testing** that surfaces issues before production.
- **The Four-Pillar Model** that decomposes agent assessment into testable components.
- **The HAAF scenario manifold** that weights evaluation by deployment risk.

The challenge is adoption. Most teams are still optimizing for the metric they can measure most easily, not the metric that matters most for production.

IX. Conclusion and Strategic Recommendations

The transition to agentic AI requires a radical departure from the evaluation methodologies of the previous generation. The current research landscape emphasizes that task completion is a necessary but insufficient condition for production readiness. Organizations and researchers must prioritize trajectory-level observability to understand why agents succeed or fail, particularly in multi-turn interactions where compounding errors are common.

The adoption of Agent-as-a-Judge paradigms offers a scalable path to achieving human-level evaluation quality while significantly reducing costs. However, this must be balanced with rigorous stress-testing of those judges using libraries like the Judge Reliability Harness to ensure they are not merely reflecting their own internal biases.

The economic dimension of agency—specifically the trade-off between accuracy and token consumption—must become a central focus of both academic research and enterprise procurement to prevent the explosion of inference bills and the deployment of operationally fragile systems.

A. Strategic Recommendations

1) For Research Teams:

1) **Report CNA alongside accuracy.** Any benchmark result reported without cost normalization is incomplete and potentially misleading.
2) **Adopt trajectory-level evaluation.** Move beyond outcome-only metrics to assess the complete execution path.
3) **Stress-test your judges.** Implement JRH-style perturbation testing for any LLM-based evaluation pipeline.

4) **Close the measurement imbalance.** Integrate human-centered and economic metrics into evaluation frameworks alongside technical performance.

2) For Engineering Teams:

1) **Instrument your traces before you instrument your benchmarks.** Trajectory observability is a prerequisite for meaningful evaluation. If you cannot replay the reasoning path for a failed task, you cannot diagnose the failure.
2) **Use multi-turn simulation to find issues before users do.** Tools like ArkSim can surface context loss, idempotency failures, and unexpected conversation paths before they appear in production logs.
3) **Apply the four pillars selectively by risk.** Not every workflow requires evaluation across all four pillars at equal depth. Triage by consequence: high-stakes workflows warrant full trajectory evaluation; lower-stakes workflows can tolerate lighter-weight assessment.
4) **Implement the CLEAR framework for deployment governance.** No agent should be promoted to production without assessment across all five CLEAR dimensions.

3) For Leadership and Procurement:

1) **Require multi-dimensional evaluation in vendor assessments.** Accuracy-only benchmarks are insufficient for procurement decisions. Demand CNA, CLEAR assessments, and trajectory-level evaluation results.
2) **Budget for evaluation infrastructure.** The cost of inadequate evaluation—production incidents, compliance violations, eroded trust—far exceeds the cost of building proper evaluation infrastructure.
3) **Invest in organizational learning.** Evaluation data should inform agent design, deployment decisions, and risk calibration across the organization.

By adopting a multi-dimensional approach that spans technical, human-centered, and economic factors, the industry can bridge the gap between impressive laboratory results and reliable, production-grade autonomous infrastructure.

© 2026 Alchedata. All rights reserved.

REFERENCES

[1] "Beyond task completion: An assessment framework for agentic ai," *arXiv preprint arXiv:2512.12791*, 2025. [Online]. Available: https://arxiv.org/html/2512.12791

[2] "Beyond benchmark islands: Toward representative trustworthiness evaluation for agentic ai," *arXiv preprint arXiv:2603.14987*, 2026. [Online]. Available: https://arxiv.org/html/2603.14987v1

[3] "What is your agent's gpa? a framework for evaluating agent goal-plan-action alignment," *arXiv preprint arXiv:2510.08847*, 2025. [Online]. Available: https://arxiv.org/html/2510.08847v2

[4] "A survey on agent-as-a-judge," *arXiv preprint arXiv:2601.05111*, 2025. [Online]. Available: https://arxiv.org/html/2601.05111v1

[5] "Agent-as-a-judge: Evaluate agents with agents," in *ICML*, 2025. [Online]. Available: https://openreview.net/forum?id=Nn9POI9Ekt

[6] "Judge reliability harness: Stress testing the reliability of llm judges," *arXiv preprint arXiv:2603.05399*, 2026. [Online]. Available: https://arxiv.org/html/2603.05399v1

[7] "The measurement imbalance in agentic ai evaluation undermines industry productivity claims," *arXiv preprint arXiv:2506.02064*, 2025. [Online]. Available: https://arxiv.org/html/2506.02064v2

[8] "Beyond accuracy: A multi-dimensional framework for evaluating enterprise agentic ai systems," *arXiv preprint arXiv:2511.14136*, 2025. [Online]. Available: https://arxiv.org/html/2511.14136v1

[9] Bessemer Venture Partners, "Securing ai agents: The defining cybersecurity challenge of 2026," 2026. [Online]. Available: https://www.bvp.com/atlas/securing-ai-agents-the-defining-cybersecurity-challenge-of-2026

[10] Galileo AI, "Agent evaluation framework 2026: Metrics, rubrics & benchmarks," 2026. [Online]. Available: https://galileo.ai/blog/agent-evaluation-framework-metrics-rubrics-benchmarks

[11] DataRobot, "Production-ready agentic ai: Evaluation, monitoring, and governance," 2026. [Online]. Available: https://www.datarobot.com/blog/production-ready-agentic-ai-evaluation-monitoring-governance/

[12] "A comprehensive empirical evaluation of agent frameworks on code-centric software engineering tasks," *arXiv preprint arXiv:2511.00872*, 2025. [Online]. Available: https://arxiv.org/html/2511.00872v1

[13] "Benchagents: Multi-agent systems for structured benchmark creation," *arXiv preprint arXiv:2410.22584*, 2025. [Online]. Available: https://arxiv.org/html/2410.22584v2

[14] AWS, "Evaluating ai agents for production: A practical guide to strands evals," 2026. [Online]. Available: https://aws.amazon.com/blogs/machine-learning/evaluating-ai-agents-for-production-a-practical-guide-to-strands-evals/

[15] ——, "Evaluating ai agents: Real-world lessons from building agentic systems at amazon," 2026. [Online]. Available: https://aws.amazon.com/blogs/machine-learning/evaluating-ai-agents-real-world-lessons-from-building-agentic-systems-at-amazon/

[16] MIT AI Agent Index, "The 2025 ai agent index: Documenting technical and safety features of deployed agentic ai systems," *arXiv preprint arXiv:2602.17753*, 2025. [Online]. Available: https://arxiv.org/html/2602.17753v1

[17] McKinsey, "One year of agentic ai: Six lessons from the people doing the work," 2026. [Online]. Available: https://www.mckinsey.com/capabilities/quantumblack/o

ur-insights/one-year-of-agentic-ai-six-lessons-from-the
-people-doing-the-work

Fei Wang Fei Wang is with Alchedata AI. Author biography details can be expanded here before final IEEE Access submission.

Salon Ren Salon Ren is with Alchedata AI. Author biography details can be expanded here before final IEEE Access submission.

Eric Wang Eric Wang is with Alchedata AI. Author biography details can be expanded here before final IEEE Access submission.